DSA0603 Data Handling and Visualization

Slot B

CO2 – Assessment Test 5 (AT2)

Visualization Development Exercise

Instructions:

For each question, write the program in the specified language (R or Python), generate the required visualization, and answer the interpretation sub-questions clearly with justification.

Question 1 — Chart Selection (Topic 1: Directory of Visualizations)

Task: For each scenario below, name the single best chart type from the directory (Amounts / Distribution / Proportion / x–y Relationship / Geospatial / Network / Temporal / Multivariate) and justify your choice in 1–2 sentences. No code required for this question.

1. Comparing total revenue across 6 product categories in one year.
2. Showing how the salary distribution differs across 4 job levels.
3. Showing market share of 5 competing brands.
4. Showing the relationship between advertising spend and sales revenue.
5. Showing monthly website traffic over 3 years.
6. Showing which employees report to which managers in an org.

Scenario	Best Chart Type	Justification
1. Comparing total revenue across 6 product categories in one year	Bar Chart (Amounts)	A bar chart is best for comparing values across discrete categories. It clearly shows which product category has the highest and

		lowest revenue.
2. Showing how the salary distribution differs across 4 job levels	Box Plot (Distribution)	A box plot summarizes the distribution using median, quartiles, and outliers, making comparison across job levels easy.
3. Showing market share of 5 competing brands	Pie Chart (Proportion)	A pie chart effectively displays how each brand contributes to the whole market share.
4. Showing the relationship between advertising spend and sales revenue	Scatter Plot (x–y Relationship)	A scatter plot clearly shows the relationship or correlation between advertising spend and sales revenue.
5. Showing monthly website traffic over 3 years	Line Chart (Temporal)	A line chart is ideal for time-series data, making trends and seasonal patterns easy to identify.
6. Showing which employees report to which managers in an organization	Network/Tree Diagram (Network)	A network or hierarchical tree diagram clearly represents reporting relationships between employees and managers.

Question 2

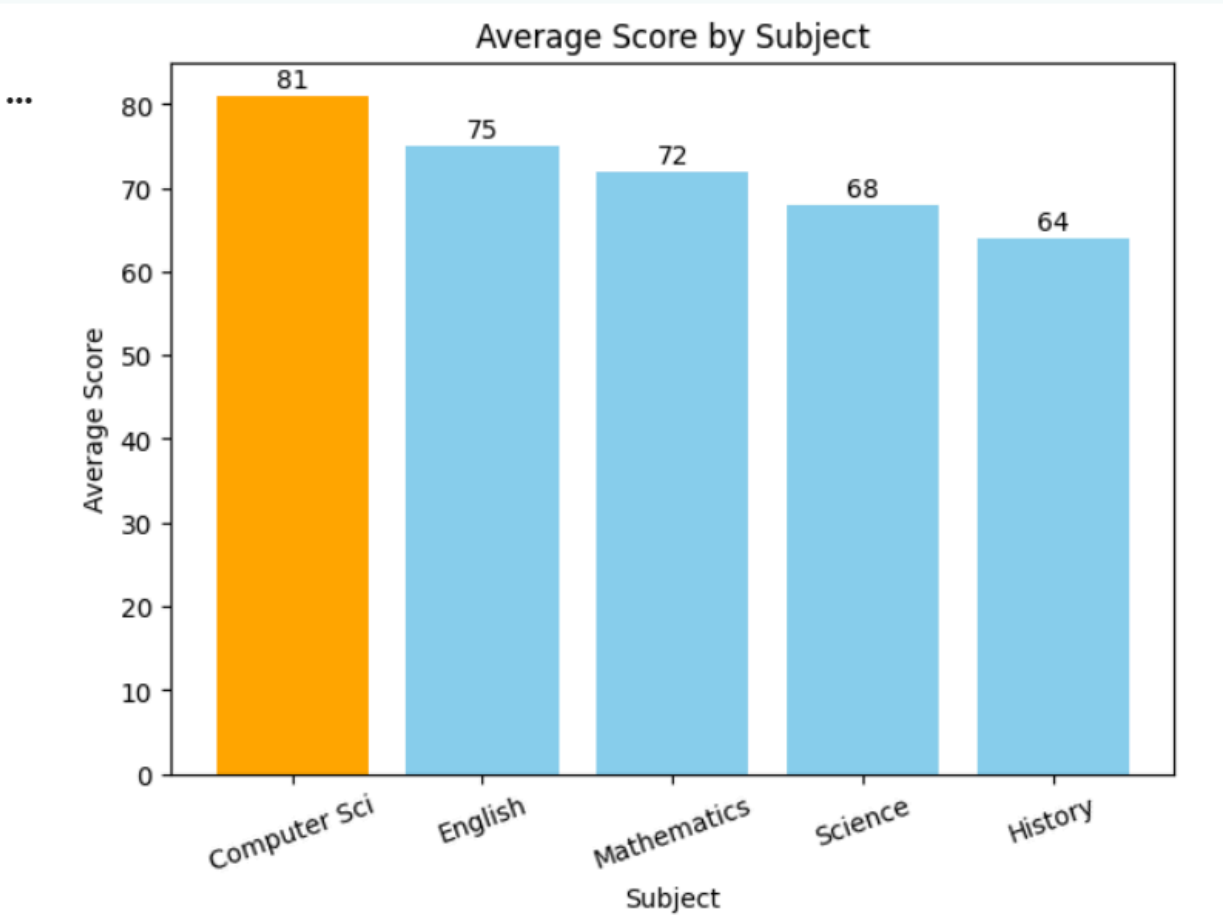
Dataset — avg_score.csv

Subject	Average_Score
Mathematics	72
Science	68
English	75
History	64
Computer Sci	81

Python code:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
   "Subject":["Mathematics","Science","English","History","Computer Sci"],
   "Average_Score":[72,68,75,64,81]
})

df = df.sort_values("Average_Score", ascending=False)

colors = ["orange" if x==df["Average_Score"].max() else "skyblue"
          for x in df["Average_Score"]]

plt.figure(figsize=(7,5))
```

```
bars = plt.bar(df["Subject"], df["Average_Score"], color=colors)

for bar in bars:
    plt.text(bar.get_x()+bar.get_width()/2,
             bar.get_height()+1,
             int(bar.get_height()),
             ha='center')

plt.title("Average Score by Subject")
plt.xlabel("Subject")
plt.ylabel("Average Score")
plt.xticks(rotation=20)
plt.show()
```

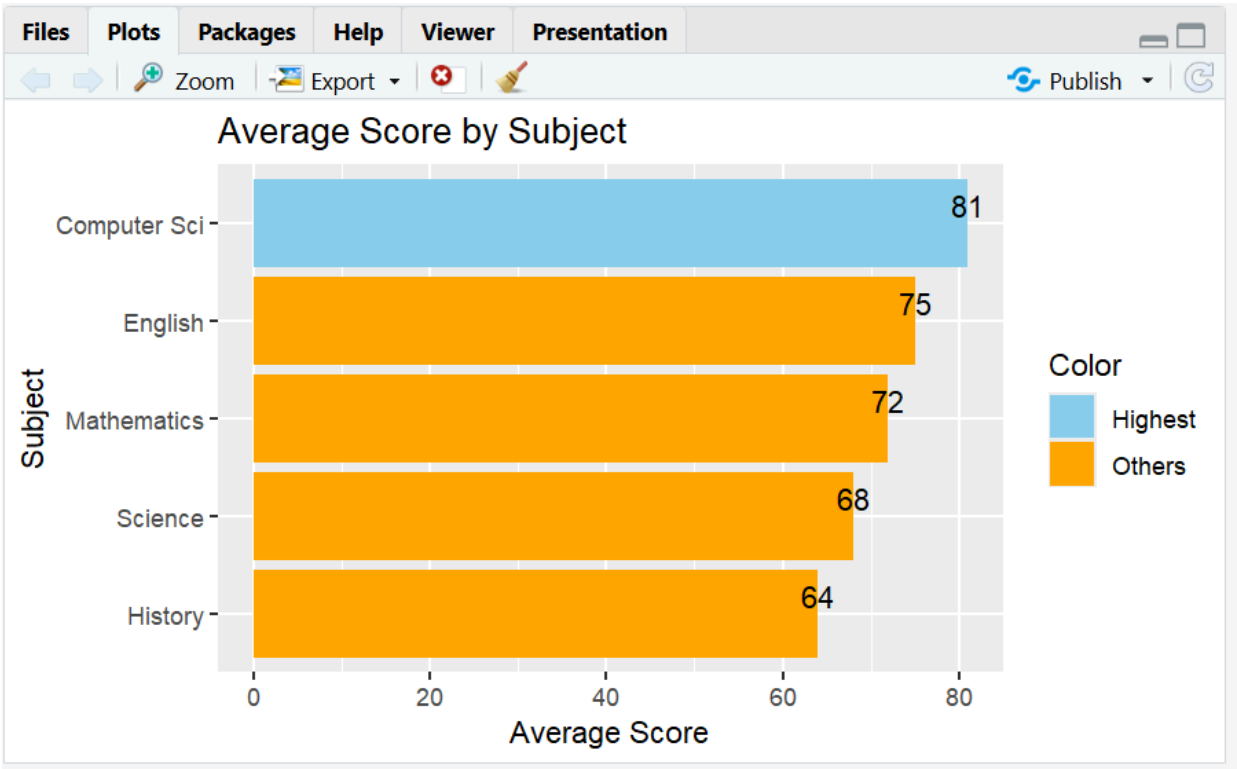


R Code:

```
library(ggplot2)

df <- data.frame(
Subject=c("Mathematics","Science","English","History","Computer Sci"),
Average_Score=c(72,68,75,64,81)
)

df <- df[order(-df$Average_Score),]
df$Color <- ifelse(df$Average_Score==max(df$Average_Score),"Highest","Others")

ggplot(df,aes(reorder(Subject,Average_Score),Average_Score,fill=Color))+
```

```
geom_col()+
geom_text(aes(label=Average_Score),vjust=-0.3)+
scale_fill_manual(values=c("skyblue","orange"))+
coord_flip()+
labs(title="Average Score by Subject",
x="Subject",
y="Average Score")
```

Output:



Task:

- Create a vertical bar plot of Average_Score by Subject, sorted in descending order of score. ● Color the bars using a single consistent color; highlight the highest-scoring subject in a different color. ● Add data labels on top of each bar.

Python: use pandas to build the DataFrame and matplotlib.pyplot.bar (or seaborn.barplot). **R:** use a data.frame and ggplot2::geom_col().

Question 3

Dataset — regional_sales.csv

Region	Quarter	Sales_000s
North	Q1	120
North	Q2	135
North	Q3	128
North	Q4	150
South	Q1	95
South	Q2	110
South	Q3	105
South	Q4	130
East	Q1	140
East	Q2	138
East	Q3	145
East	Q4	160
West	Q1	80
West	Q2	85

West	Q3	90
West	Q4	100

Task (produce all four views from the same dataset):

1. Grouped bar chart: Sales_000s by Region, grouped by Quarter.

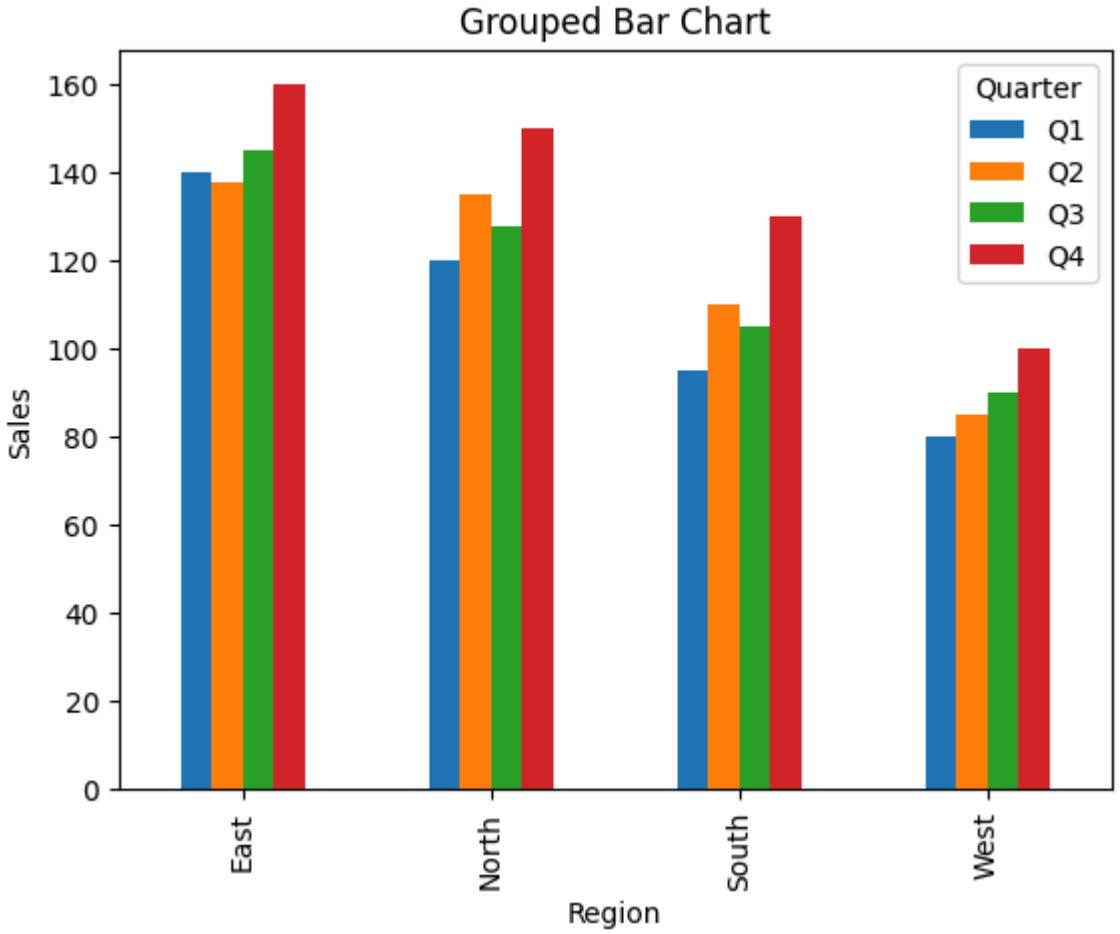
2. Stacked bar chart: same data, stacked by Quarter within each Region.

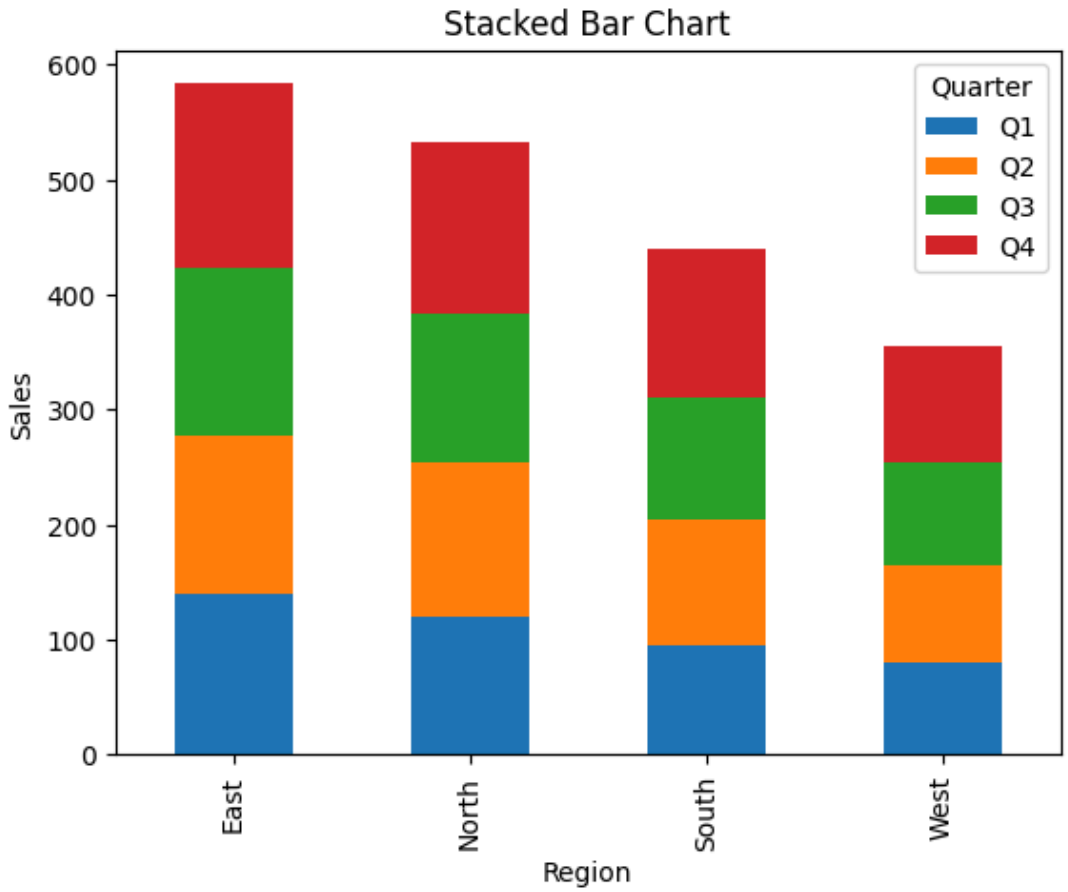
3. Dot plot: total annual sales per region (sum across quarters), ranked from highest to lowest. 4. Heatmap: Region (rows) × Quarter (columns), color-encoded by Sales_000s.

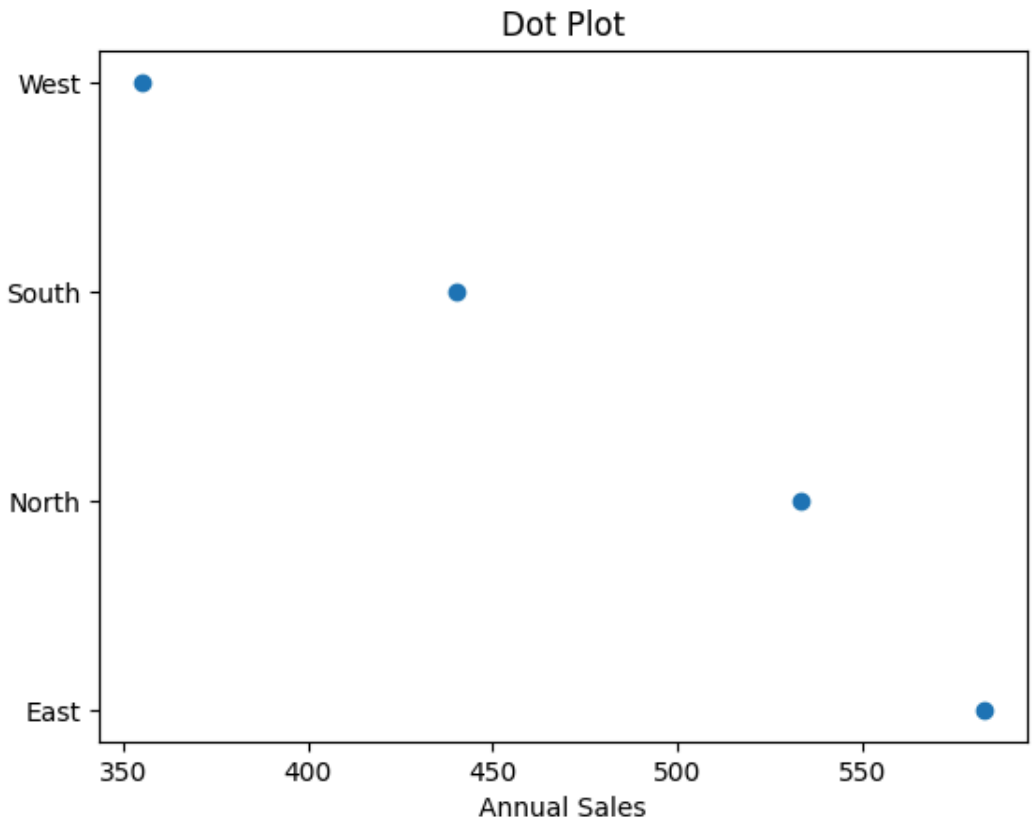
Python: pandas.pivot_table to reshape; matplotlib/seaborn for grouped/stacked bars, seaborn.heatmap for the heatmap, matplotlib.pyplot.scatter (or plt.plot with markers) for the dot plot.

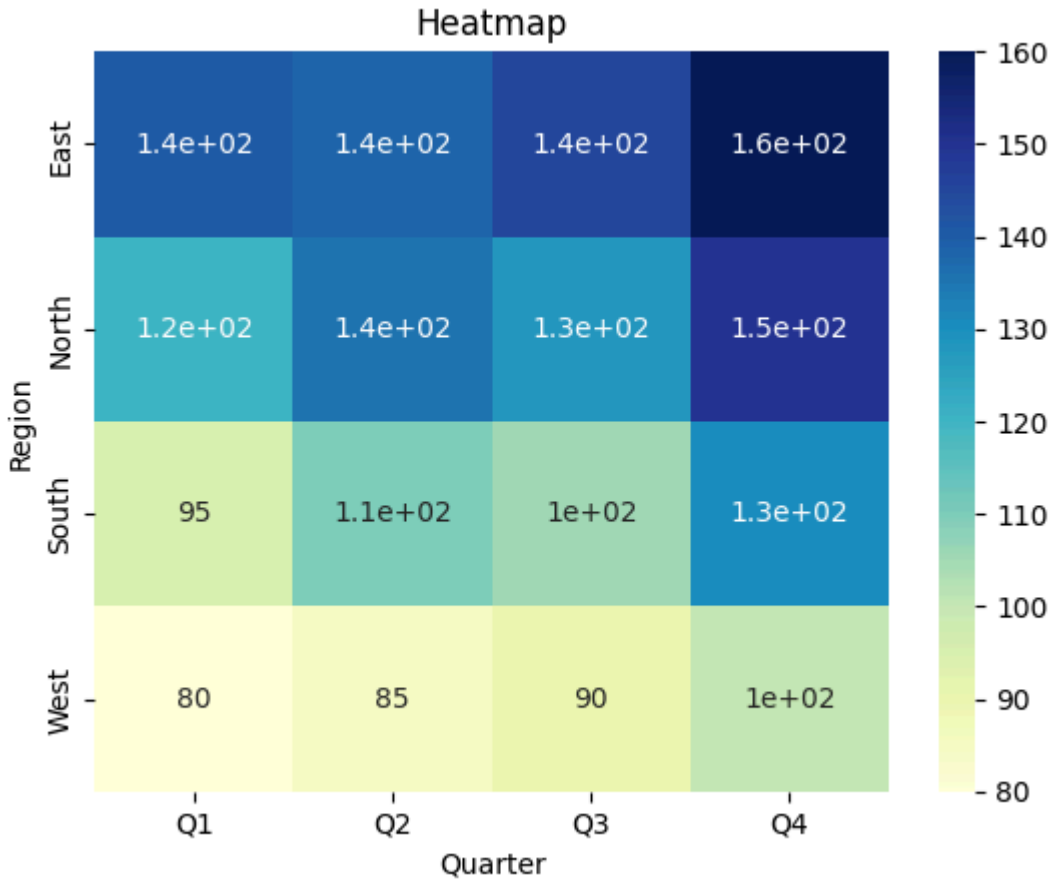
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.DataFrame({
"Region":["North","North","North","North",
"South","South","South","South",
"East","East","East","East",
"West","West","West","West"],
"Quarter":["Q1","Q2","Q3","Q4"]*4,
"Sales":[120,135,128,150,
95,110,105,130,
140,138,145,160,
80,85,90,100]
```

```
})

pivot = df.pivot(index="Region",columns="Quarter",values="Sales")

pivot.plot(kind="bar")
plt.title("Grouped Bar Chart")
plt.ylabel("Sales")
plt.show()

pivot.plot(kind="bar",stacked=True)
plt.title("Stacked Bar Chart")
plt.ylabel("Sales")
plt.show()

total = df.groupby("Region")["Sales"].sum().sort_values(ascending=False)

plt.scatter(total.values,total.index)
plt.title("Dot Plot")
plt.xlabel("Annual Sales")
plt.show()

sns.heatmap(pivot,annot=True,cmap="YlGnBu")
plt.title("Heatmap")
plt.show()
```


Grouped Bar Chart
Quarter
Q1
Q2
Q3
Q4
Sales
160
140
120
100
80
60
40
20
0
East
North
South
West
Region


Stacked Bar Chart
Quarter
Q1
Q2
Q3
Q4
Sales
600
500
400
300
200
100
0
East
North
South
West
Region


Dot Plot
West
South
North
East
350
400
450
500
550
Annual Sales



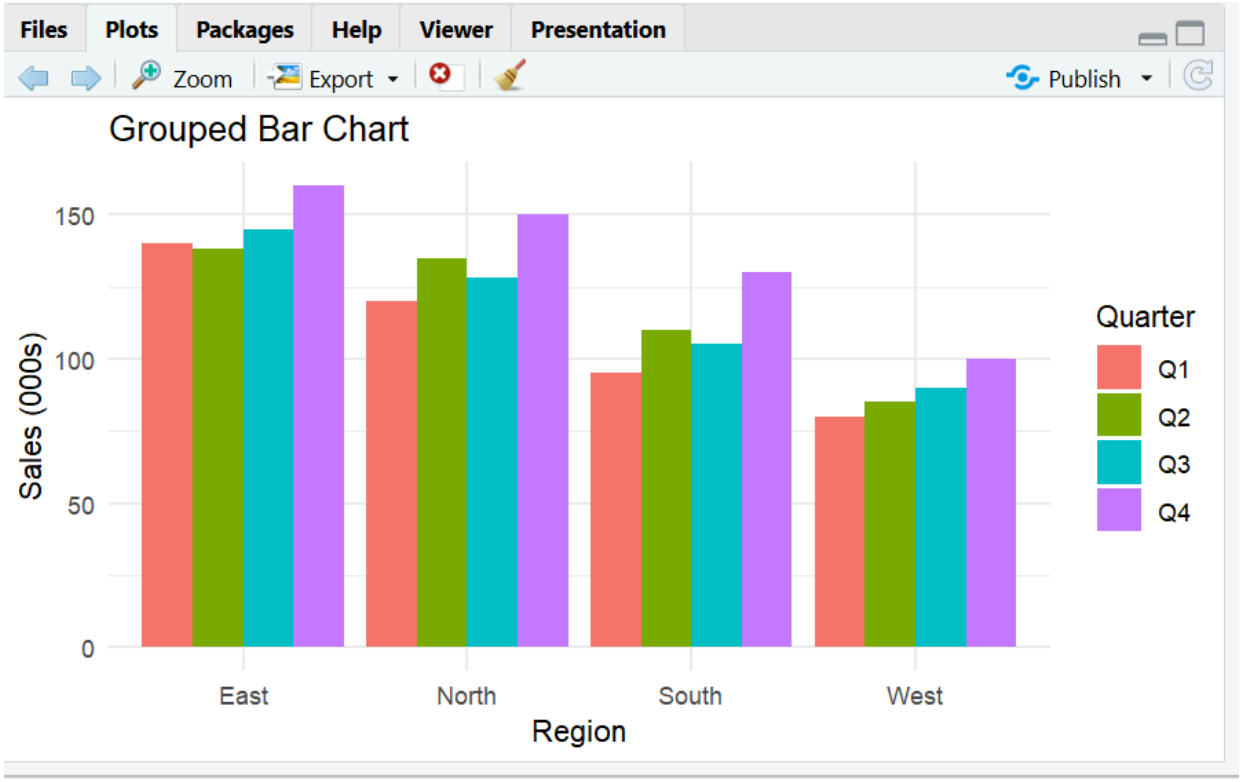
R: tidyr::pivot_wider where needed; ggplot2::geom_col(position = "dodge") for grouped, position = "stack" for stacked, geom_point() for the dot plot, geom_tile() for the heatmap.

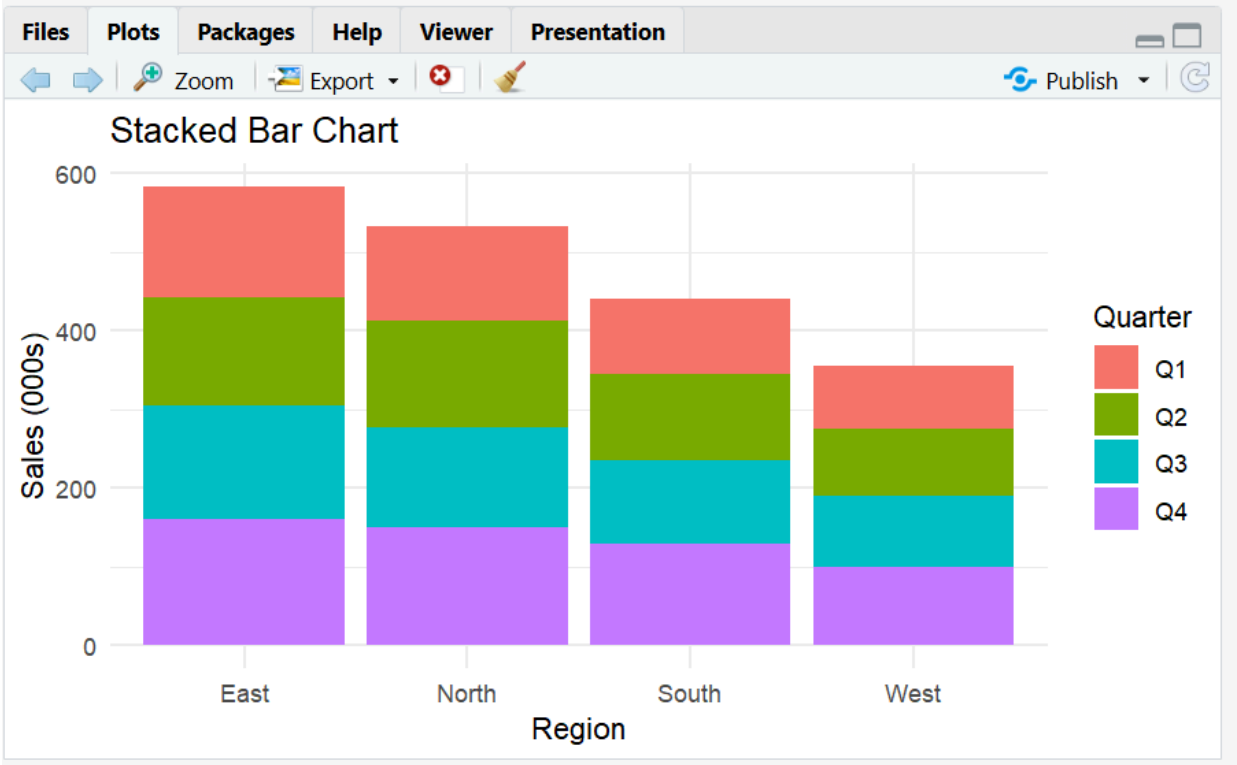
```
# Install packages if needed

# install.packages("ggplot2")


library(ggplot2)


# Create Dataset

df <- data.frame(
```

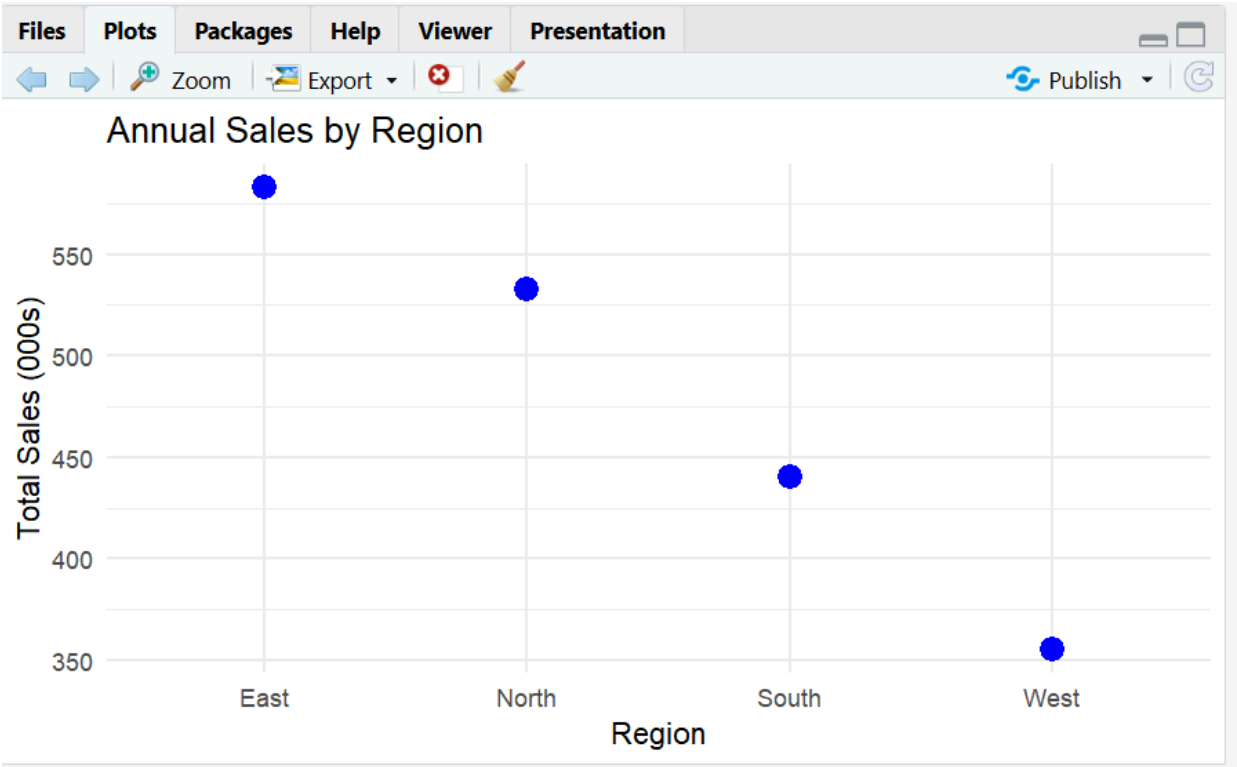
```
  Region = rep(c("North", "South", "East", "West"), each = 4),
  Quarter = rep(c("Q1", "Q2", "Q3", "Q4"), 4),
  Sales = c(
    120, 135, 128, 150,
    95, 110, 105, 130,
    140, 138, 145, 160,
    80, 85, 90, 100
  )
)
p1 <- ggplot(df, aes(x = Region, y = Sales, fill = Quarter)) +
  geom_col(position = "dodge") +
  labs(
    title = "Grouped Bar Chart",
    x = "Region",
    y = "Sales (000s)"
  ) +
  theme_minimal()
print(p1)
p2 <- ggplot(df, aes(x = Region, y = Sales, fill = Quarter)) +
  geom_col(position = "stack") +
  labs(
    title = "Stacked Bar Chart",
```

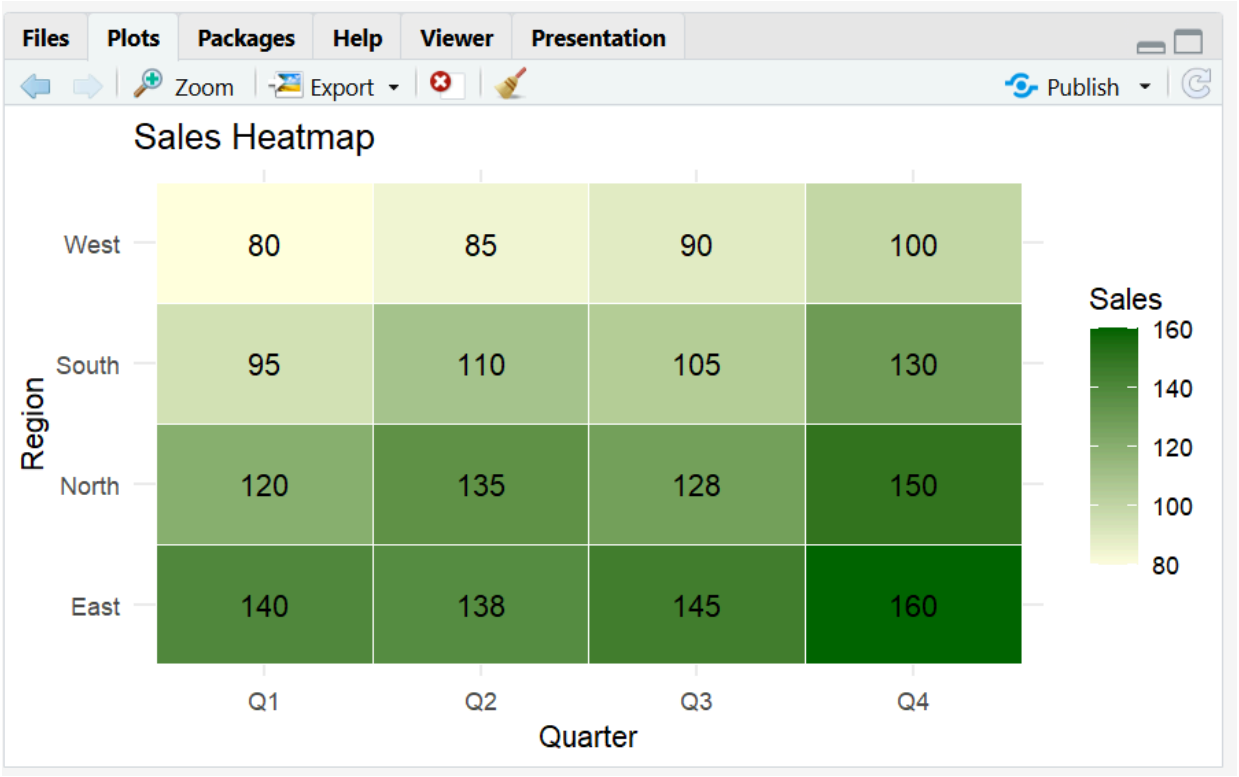
```
    x = "Region",
    y = "Sales (000s)"
  ) +
  theme_minimal()
print(p2)
tot <- aggregate(Sales ~ Region, data = df, sum)
tot <- tot[order(-tot$Sales), ]
p3 <- ggplot(tot, aes(x = reorder(Region, -Sales), y = Sales)) +
  geom_point(size = 4, color = "blue") +
  labs(
    title = "Annual Sales by Region",
    x = "Region",
    y = "Total Sales (000s)"
  ) +
  theme_minimal()
print(p3)
# 4. Heatmap
p4 <- ggplot(df, aes(x = Quarter, y = Region, fill = Sales)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Sales), color = "black") +
  scale_fill_gradient(low = "lightyellow", high = "darkgreen") +
  labs(
```

```
  title = "Sales Heatmap",
  x = "Quarter",
  y = "Region"
 ) +
 theme_minimal()
print(p4)
```




Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Stacked Bar Chart
600
400
200
0
Sales (000s)
East
North
South
West
Region
Quarter
Q1
Q2
Q3
Q4


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Annual Sales by Region
550
500
450
400
350
Total Sales (000s)
East
North
South
West
Region



Question 4

Dataset — generate synthetically for reproducibility:

Class A: 40 exam scores ~ Normal(mean=70, sd=8)

Class B: 40 exam scores ~ Normal(mean=75, sd=5)

Class C: 40 exam scores ~ Normal(mean=65, sd=12)

Class D: 40 exam scores ~ Normal(mean=80, sd=6)

(Set a random seed of 42 in both Python and R so results are consistent across languages.) **Task:**

1. Violin plot: exam score distribution by class (all 4 classes on one plot), with an

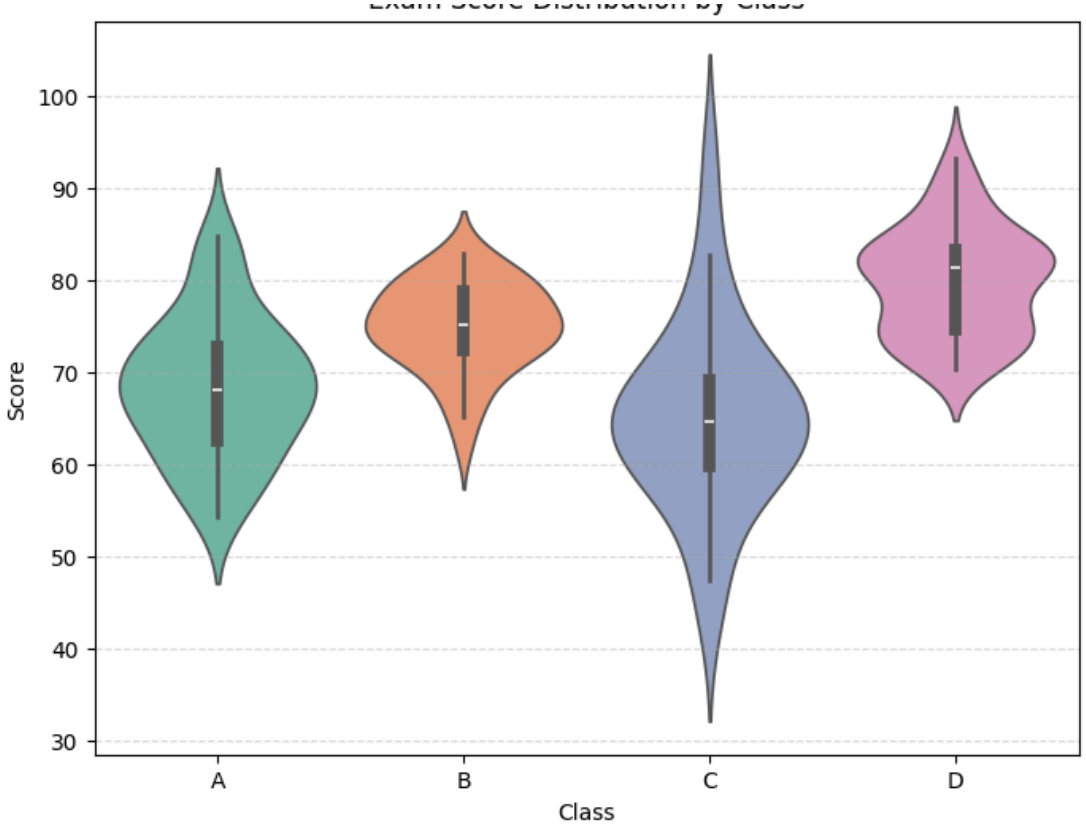
embedded boxplot or median marker.

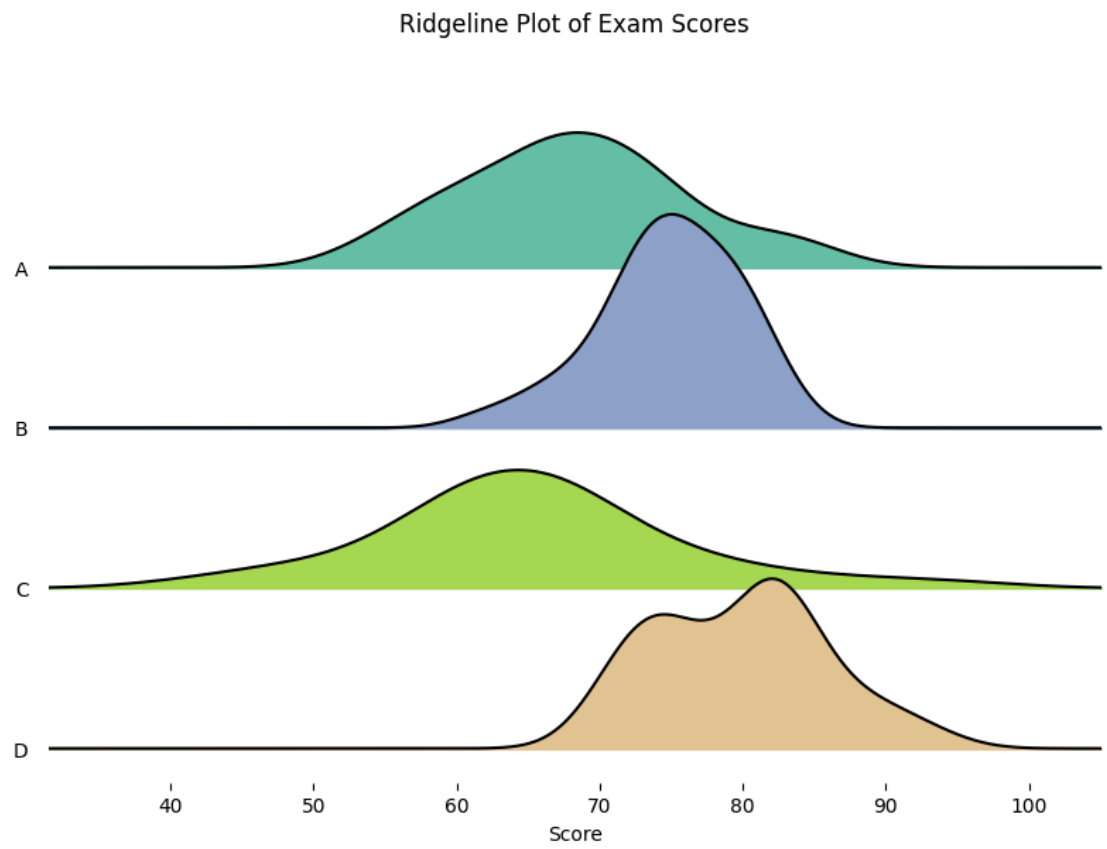
2. Ridgeline plot: same 4 classes, overlapping density curves, ordered by mean score.

Python: numpy.random.normal to generate data, seaborn.violinplot for the violin plot, joypy.joyplot (or a stacked seaborn/matplotlib kdeplot FacetGrid) for the ridgeline.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import joypy
np.random.seed(42)
data=[]
classes={
"A":(70,8),
"B":(75,5),
"C":(65,12),
"D":(80,6)
}

for c,(m,s) in classes.items():
    scores=np.random.normal(m,s,40)
    for x in scores:
        data.append([c,x])

df=pd.DataFrame(data,columns=["Class","Score"])
```

```
sns.violinplot(data=df,x="Class",y="Score",inner="box")
plt.title("Violin Plot")
plt.show()

joypy.joyplot(df,by="Class",column="Score")
plt.title("Ridgeline Plot")
plt.show()
```

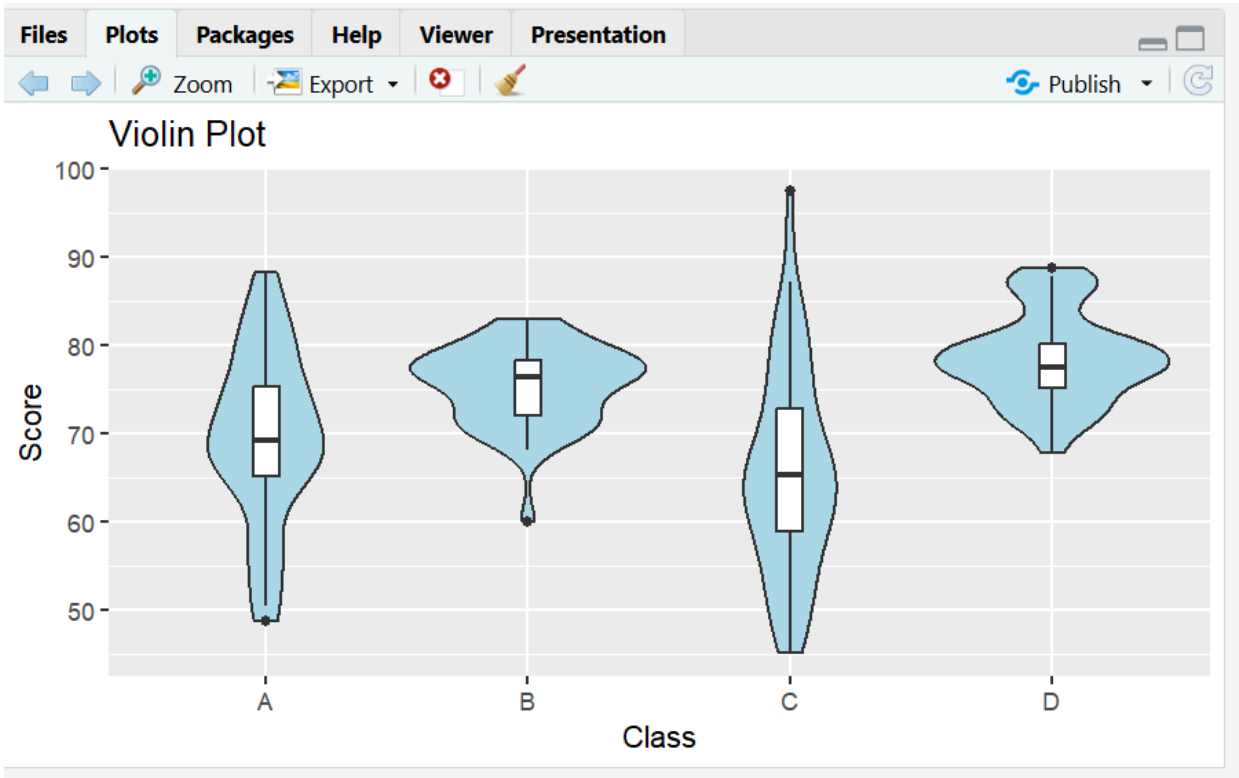


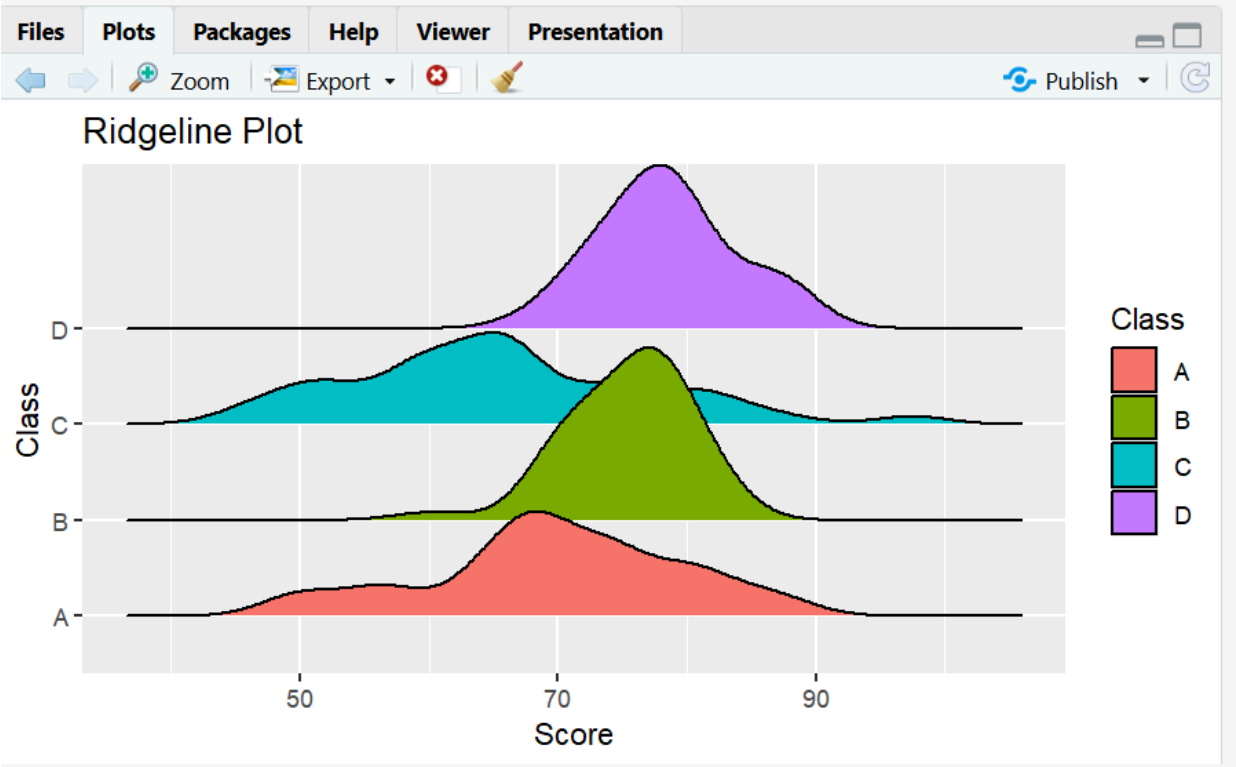


R: rnorm() to generate data, ggplot2::geom_violin() for the violin plot, ggridges::geom_density_ridges() for the ridgeline.

```
library(ggplot2)

library(ggridges)

set.seed(42)

df<-data.frame(

Class=rep(c("A","B","C","D"),each=40),

Score=c(rnorm(40,70,8),

rnorm(40,75,5),
```

```
rnorm(40,65,12),
rnorm(40,80,6))
)

ggplot(df,aes(Class,Score))+
geom_violin(fill="lightblue")+
geom_boxplot(width=.1)+
labs(title="Violin Plot")

ggplot(df,aes(Score,Class,fill=Class))+
geom_density_ridges()+
labs(title="Ridgeline Plot")
```


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Violin Plot
Score
100
90
80
70
60
50
A
B
C
D
Class



Question 5

Dataset: Use a built-in dataset available in both languages.

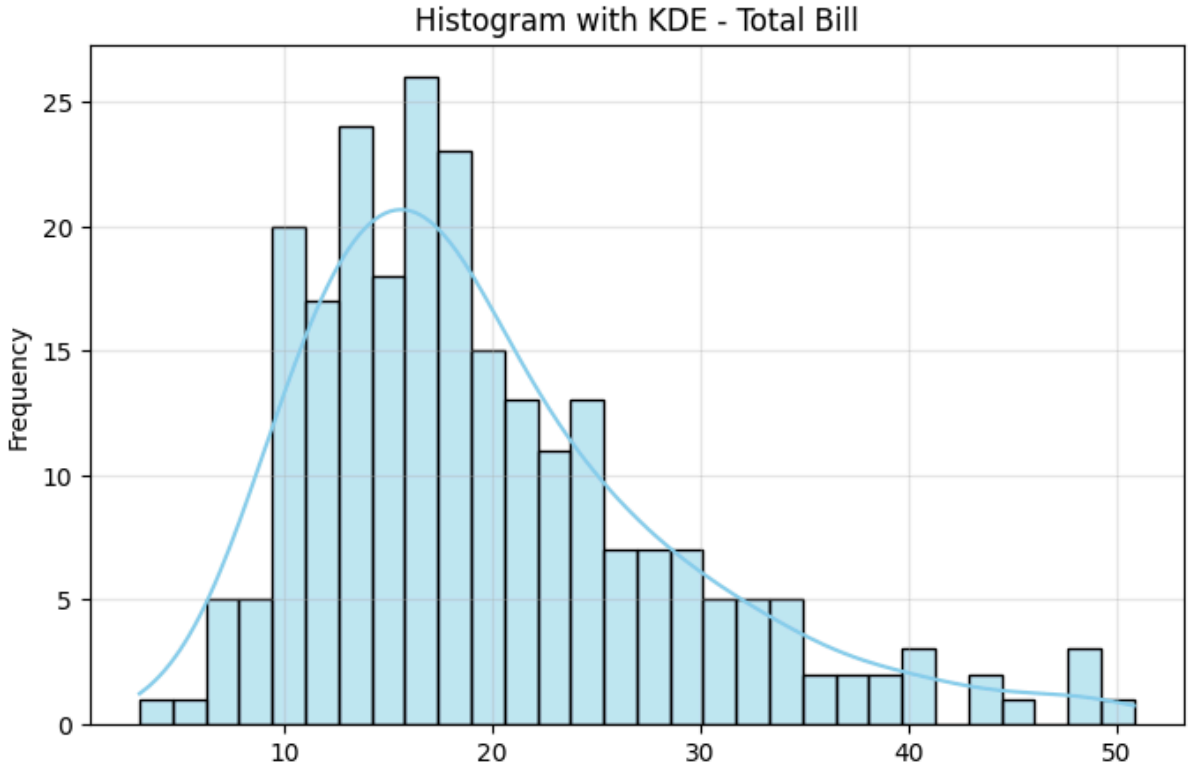
- Python: seaborn.load_dataset("tips") → use the total_bill column.
- R: the equivalent tips dataset (available via the reshape2 package, or import the same CSV used in Python so results are directly comparable).

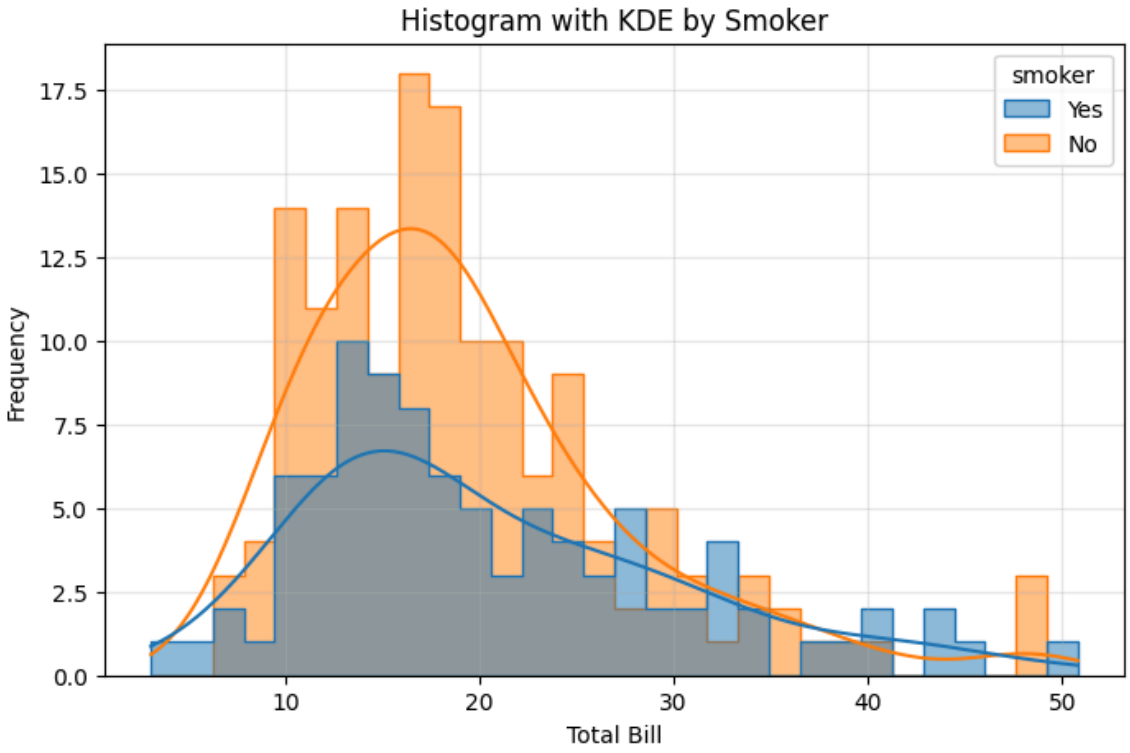
Task:

1. Plot a histogram of the chosen continuous variable with 30 bins.
2. Overlay a kernel density estimate (KDE) curve on the same plot.
3. Repeat both, split/colored by the smoker categorical variable, to compare distribution shape across groups. **Python:** seaborn.histplot(..., kde=True) or matplotlib.pyplot.hist + scipy.stats.gaussian_kde.

import seaborn as sns

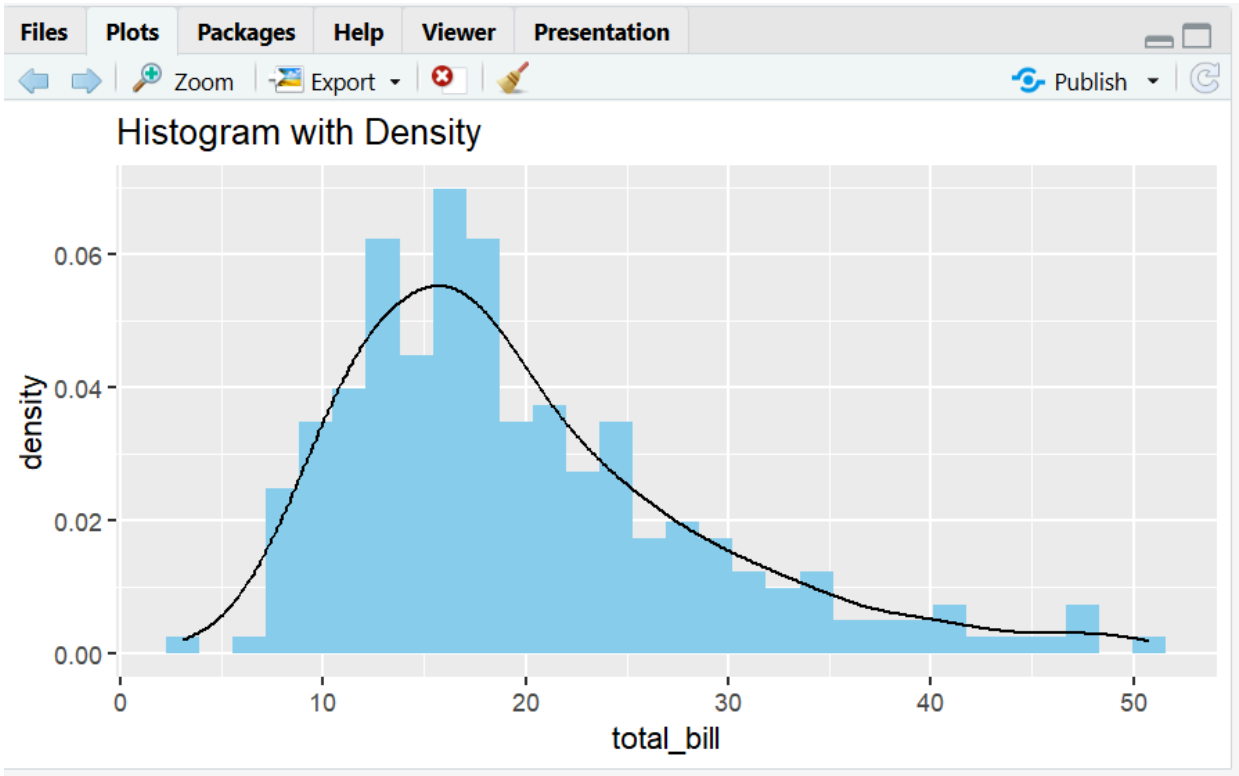
```
import matplotlib.pyplot as plt
tips=sns.load_dataset("tips")
sns.histplot(tips["total_bill"],bins=30,kde=True)
plt.title("Histogram with KDE")
plt.xlabel("Total Bill")
plt.show()
sns.histplot(data=tips,
x="total_bill",
hue="smoker",
bins=30,
kde=True)
plt.title("Histogram by Smoker")
plt.show()
```

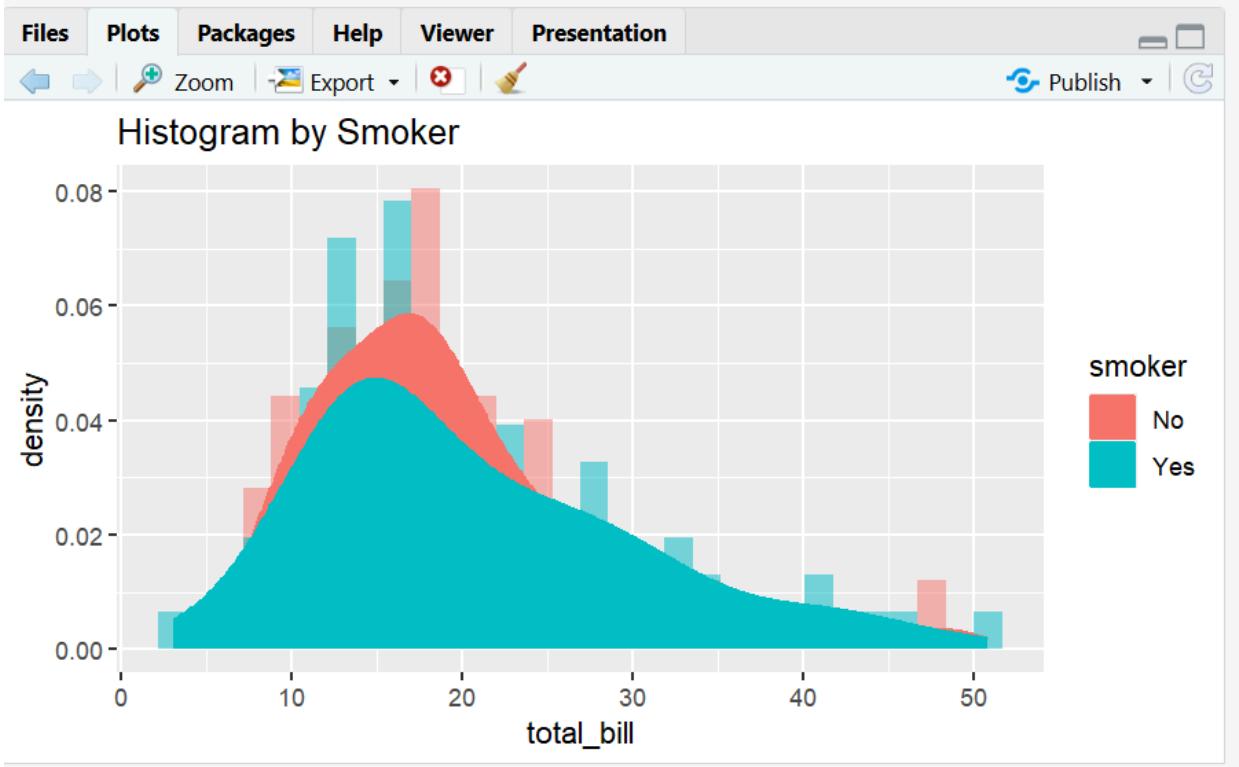

Histogram with KDE - Total Bill
Frequency
0
5
10
15
20
25
10
20
30
40
50


Histogram with KDE by Smoker
smoker
Yes
No
Frequency
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
10
20
30
40
50
Total Bill

R: ggplot2::geom_histogram() combined with geom_density() (use after_stat(density) so scales match), with fill = mapped to smoker for the grouped version.

```
library(ggplot2)
library(reshape2)
data(tips)
ggplot(tips,aes(total_bill))+
geom_histogram(aes(y=after_stat(density)),
bins=30,
fill="skyblue")+
geom_density()+
labs(title="Histogram with Density")

ggplot(tips,aes(total_bill,fill=smoker))+
geom_histogram(aes(y=after_stat(density)),
bins=30,
alpha=0.5,
position="identity")+
geom_density(aes(color=smoker))+
labs(title="Histogram by Smoker")
```


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Histogram with Density
density
0.06
0.04
0.02
0.00
0
10
20
30
40
50
total_bill


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Histogram by Smoker
density
0.08
0.06
0.04
0.02
0.00
0
10
20
30
40
50
total_bill
smoker
No
Yes

Submission Checklist

- Q1: written chart-type justifications (no code)
- Q2: bar plot — Python + R
- Q3: grouped bar, stacked bar, dot plot, heatmap — Python + R
- Q4: violin plot, ridgeline plot — Python + R
- Q5: histogram + density (overall and grouped) — Python + R
- Each chart has a title, axis labels, and a short written interpretation